BIOMASS MAAP HACKATHON 2026 · KICK-OFF

Using the repo
good coding practices

BioPAL

Yoann Rey-Ricord

TODAY'S PLAN

I

THE REPOSITORY

Fork, branch, pull request. Ground rules.

II

CODE FOR REUSE

Modular, documented, light tooling.

III

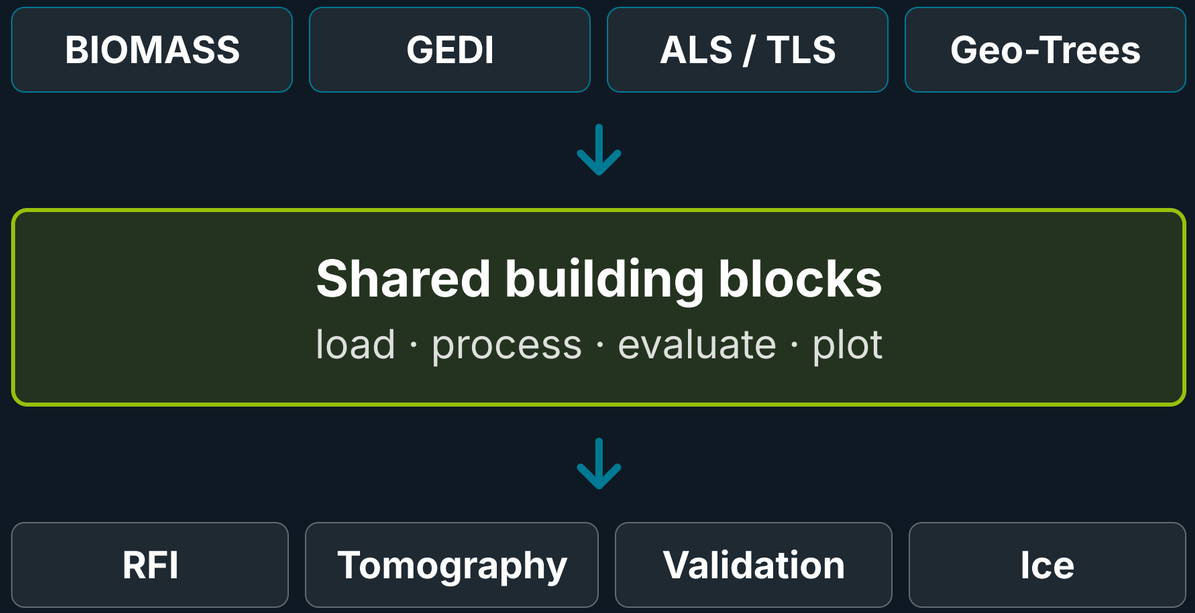
YOUR CHECKLIST

What to leave behind on Friday.

WHY THIS MATTERS

Many datasets, one toolbox

- **10 topics** use the same products: BIOMASS, GEDI, ALS, TLS, Geo-Trees.
- **Modular** code lets another group reuse it on Wednesday.
- **Documented** code still runs in six months.



WHAT YOU WILL FIND

One folder per topic

```
biomass-hackathon-2026/
├── README.md
├── CONTRIBUTING.md
├── resources/        # shared .py modules
│   ├── load/
│   ├── process/
│   ├── evaluate/
│   ├── plot/
│   └── tests/
└── topics/
    ├── 01-polsar-polinsar-analytics/
    ├── 04-3d-forest-structure/
    ├── 06-biomass-validation/
    ├── …                  # one folder per challenge
    └── 12-forest-height-tomosar/
```

- Your folder has a **README**: goal, roadmap, data.
- Keep its status up to date.
- Shared code → `resources/` (**load**, **process**, **evaluate**, **plot**, **tests**).
- **Link** to data, never commit it.

HOW YOU CONTRIBUTE

Fork, branch, pull request

1

Fork on GitHub

Fork `BioPAL/biomass-hackathon-2026` (button on github.com)

2

Clone your fork

Clone `YOU/biomass-hackathon-2026`, not BioPAL directly

3

Branch and push

One branch per topic, push to your fork

4

Pull request

From your fork into BioPAL `main`

`main` is protected. Never merge your own PR. Ask anyone in the room for Git help all week.

THE COMMANDS

Fork first, then git

```
# 1. On GitHub (once): Fork BioPAL/biomass-hackathon-2026
#    → your copy: github.com/YOU/biomass-hackathon-2026

# 2. On your laptop (once): clone YOUR fork only
git clone https://github.com/YOU/biomass-hackathon-2026.git
cd biomass-hackathon-2026
git remote add upstream https://github.com/BioPAL/biomass-hackathon-2026.git

# 3. Per task
git checkout -b 04-3d-forest-structure/tomo-processor
git add topics/04-3d-forest-structure/
git commit -m "Add tomographic processor notebook"
git push -u origin 04-3d-forest-structure/tomo-processor
```

→ Then open the PR on github.com: `head` = your fork and branch, `base` = `BioPAL/biomass-hackathon-2026` · `main`.

GROUND RULES

Four rules for everyone

No big data

Link, don't commit

MAAP link or a small download script.

No secrets

Ever

No keys, tokens or passwords in the repo.

Your folder

Share via resources/

Edit your topic folder only.

Apache 2.0

Open by default

Use compatible licences for code and data.

THIS WEEK

How a change gets merged

You

Open a pull request from your fork to `main`

→

Maintainer

Quick review, then merge. No CI gate this week.

Keep it simple: small PRs, clear title, say what changed. Stricter rules for upstream BPS are in the appendix.

II

CODE FOR REUSE

Modular · documented · light tooling

MODULAR

Four layers, kept apart

Load

Read BIOMASS, GEDI, ALS into arrays.

→

Process

Arrays in, arrays out.

→

Evaluate

Statistics and validation.

→

Present

Plots, maps, report.

New dataset? Change **Load** only. The rest keeps working.

MODULAR

Rules that make code reusable

1. **One function, one job.**
2. **Parameters,** not constants: paths, bands, thresholds.
3. **Logic in .py files.** Notebooks only call it and show results.
4. **No plotting** inside processing functions.

> Can someone call it on **their** dataset in three lines?
>
> If yes, it is reusable.

BioPAL

</> MODULAR

From notebook cell to function

✕ Hard to reuse

```
# notebook cell
img = rasterio.open(
    "/Users/me/L2A.tif").read(1)
fh = np.where(img > 0, img*.5, np.nan)
plt.imshow(fh)
rmse = np.sqrt(
    np.mean((fh - als) ** 2))
```

Hard-coded path, mixed steps

✓ Reusable

```
# src/metrics.py
def rmse(pred: np.ndarray,
         ref: np.ndarray) → float:
    """RMSE [m], NaN ignored."""
    ok = np.isfinite(pred + ref)
    return float(np.sqrt(np.mean(
        (pred[ok] - ref[ok]) ** 2)))
```

Typed, documented, testable

DOCUMENT

Three things, every time

- **README** per topic: what, how to run, data.
- **Docstring** on shared functions, with units.
- **Comment** the why, not the what.

Notebooks: one markdown line before each step.

```
# 06 · Validation

## What it does
BIOMASS height vs Geo-Trees.

## How to run
pip install -r requirements.txt
jupyter lab notebooks/01.ipynb

## Needs
BIOMASS L2A (path in config.yaml)

## Status
1 site working.
```

REPRODUCIBLE

Anyone can rerun it

Dependencies

requirements.txt

List what you import.

No fixed paths

Config or arguments

Never /Users/me in code.

Fetch data

Small script

Document how to get MAAP data.

Fixed seeds

ML and sampling

Record versions and parameters.

NOTEBOOKS

Notebooks tell, modules do

- **Restart and run all** before you commit.
- Clear heavy outputs.
- Copied a cell twice? Make it a function.
- Number them: `01_load.ipynb`

```
topic/
├── README.md
├── requirements.txt
├── notebooks/
│   ├── 01_load.ipynb
│   └── 02_report.ipynb
└── src/
    ├── io.py
    ├── metrics.py
    └── report.py
```

LOCAL TOOLS

Two minutes, no more

```
# once
pip install ruff pytest

# before each commit
ruff check --fix topics/XX
ruff format topics/XX

# optional, one test
pytest -q topics/XX
```

- **ruff** lints, sorts imports, formats.
- **pytest** on a tiny array protects your core function.
- Noisy on a notebook? Skip it. Keep moving.

Same tools as BPS. mypy and pre-commit are optional here.

STANDARDS

The BPS style, in five lines

Names	snake_case, verbs for functions	`load_product()`
Classes	PascalCase	`BiomassLoader`
Types	hints on signatures	`x: np.ndarray → float`
Docstrings	NumPy style, with units	`Parameters / Returns`
Lines	120 characters, ruff formats	`ruff format`

GIT HABITS

Small, frequent, readable

Commit often

Small steps

One change per commit.

Say why

Clear message

Not “update” or “fix”.

Push daily

Do not lose work

Open a draft PR early.

Check first

git status, git diff

No data, no secrets.

HACKATHON REALITY

How much is enough?

Always

cheap, decisive

- ✓ Load / process / plot apart
- ✓ Docstring on shared code
- ✓ README: what, how, data
- ✓ No secrets or big data

If time allows

nice for hand-over

- ✓ ruff check and format
- ✓ Type hints
- ✓ One pytest
- ✓ Pinned requirements

Later

if it goes upstream

- ✓ Full test tiers
- ✓ mypy, pre-commit
- ✓ DCO sign-off
- ✓ Expert review

BioPAL

BEFORE FRIDAY

Ready to be reused?

- [x] Load, process, plot are separate
- [x] README: what, how to run, data
- [x] Docstrings with units on shared functions
- [x] requirements.txt, ruff check passes
- [x] Notebooks run from a fresh kernel
- [x] Pull request opened

AFTER THE WEEK

Where your code goes next

1
Your topic folder
merged by a maintainer

→

2
resources/
shared by several groups

→

3
Upstream
PyPolSARpro, BPS, QGIS, MAAP

The more modular and documented, the cheaper the next step.

GETTING HELP

Ask early

On site

Ask anyone

Other teams, referents, maintainers, organisers — all week.

Help wanted

GitHub issue

Specific question or data access.

Discussions

Ideas

Anything not tied to one task.

Topic referent

Science and data

Feel free to ask the referent of your topic.

Modular. Documented. Reusable.

github.com/BioPAL/biomass-hackathon-2026

Questions? Then let's code. Appendix follows.

A

APPENDIX

Reference material for later

REFERENCE

BPS standards and hackathon expectations

	BIOPAL/BPS	THIS HACKATHON
Branches	feature/ bugfix/ docs/ from develop	One branch per topic, PR to main
Review	CI tiers, maintainer, module expert	One maintainer review, merge to main
Commits	Signed-off (DCO), pre-commit	Clear messages
Style	ruff, type hints, NumPy docstrings	ruff check, docstrings on shared code
Tests	pytest by tier	One small test if time
Licence	Apache 2.0 compatible, REUSE	Apache 2.0 compatible

REFERENCE

Code conventions (BPS)

Variables, functions	snake_case, verbs for functions	`incidence_angle, load_product()`
Classes	PascalCase	`BiomassLoader`
Constants	UPPER_SNAKE_CASE	`DEFAULT_THRESHOLD`
Modules	snake_case	`sar_utils.py`
Type hints	on signatures	`def f(x: np.ndarray) → float`
Docstrings	NumPy style, units	`Parameters / Returns`
Line length	120 characters	`enforced by ruff`
Imports	grouped, sorted	`ruff does it`
Tests	test_ prefix	`test_rmse_ignores_nan`

REFERENCE

NumPy docstring template

```
def rmse(pred: np.ndarray, ref: np.ndarray) → float:
    """Root mean square error, NaN ignored.

    Parameters
    ----------
    pred : np.ndarray
        Estimated forest height [m].
    ref : np.ndarray
        Reference (e.g. ALS) height [m], same shape as ``pred``.

    Returns
    -------
    float
        RMSE in metres.
    """
```

REFERENCE

Useful links

Hackathon repository
`github.com/BioPAL/biomass-hackathon-2026`

BPS source code
`github.com/BioPAL/BPS`

PyPolSARpro
`polsarpro.readthedocs.io`

ruff
`docs.astral.sh/ruff`

MAAP platform
`maap-project.org`